Programmable In-Network Timing Obfuscation for DNP3

Anonymous Submission

***Abstract*—Device fingerprinting is an early step in reconnaissance against industrial control systems. On a power-grid control network, a passive observer can tell what kind of operation a DNP3 outstation is performing from the time the device takes to answer, and that timing remains visible whether or not the payload is readable. Traffic-obfuscation defenses built for general network traffic do not transfer to this setting, because they assume an endpoint that can be modified to shape its own traffic, whereas a protective relay cannot be changed and the leaking quantity is an interval inside a single request-response exchange rather than a property of a flow. Adding random jitter is also weaker than it appears, since a jittered interval retains statistical structure that an observer recovers by repeating the measurement.**

We present an in-network timing-obfuscation framework that runs in the data plane of one programmable switch placed in front of the outstation. The framework classifies each DNP3 exchange, holds the packets that carry the outstation's timing, and releases them on operator-set offsets, so the interval an observer records is a policy value instead of the device's own. The intervention is timing only: no packet changes size. We implement it as a P4 program on an Intel Tofino-1 and evaluate it against a physical SEL-751A relay over 22 grouped collection runs in one approximately five-hour campaign, comprising 63,360 DNP3 exchanges.

A measured sweep of the release policy on the switch shows that the interval an observer records follows the configured offset across a usable range while the end-to-end response time stays fixed, and that the range is closed from above by the finite fail-open horizon. Under the operating policy the cross-layer response time, the feature this class of reconnaissance relies on, is replaced by the policy value: its median moves to the configured release and its interquartile range falls from about 2.8 ms to 0.006 ms. For the evaluated fixed Random-Forest attacker, one that profiles the device beforehand and applies that model unchanged, three-class transaction identification falls from 0.651 balanced accuracy to approximately three-class chance, and the mutual information between the interval and the transaction class falls from 0.383 bits to a value inside a within-run permutation null. An attacker that instead retrains on obfuscated traffic recovers part of the signal from the difference between the read-path and control-path anchors, which bounds what the mechanism achieves. On the master-facing link the framework adds no frame and no byte for the same workload, and costs about 21 to 23 ms of added response time per exchange. These results characterise one relay behind one switch over one campaign: the plaintext function codes remain visible, a fixed release budget leaves a small tail outside the target interval, the evaluated classification is of transaction classes rather than device models, and we make no claim about distinguishing devices.

***Index Terms*—traffic obfuscation, device fingerprinting, DNP3, programmable switches.**

I. Introduction

Device fingerprinting has been an essential step in cyber reconnaissance, allowing adversaries to reveal unique features of target networks and design effective, stealthy attack strategies. These techniques are becoming increasingly critical in industrial control systems (ICSs) such as power grids, where adversaries often use IP-based control networks and computing devices within as entry points to inflict physical damage. Consequently, fingerprinting shifts the focus from visited websites and user biometric behavior to device models and types of control operations that are critical to ICS attacks. In the 2015 attack that disrupted Ukrainian power grids and the Stuxnet attack that disrupted Iranian nuclear power facilities, it is widely believed that adversaries stay in their systems for at least 6 months to perform cyber reconnaissance [1], [2].

To disrupt device fingerprinting, many studies present network traffic obfuscation [3], [4]. Because device fingerprinting targeting general computing environments generally relies on network-level features such as packet size and/or inter-packet latency observed from communication patterns [5], traffic obfuscation often focuses on (i) padding and splitting network packets, which hide or change the distribution of network packet sizes [3], and (ii) delaying network packets and adding dummy ones, which disrupt the inter-packet timing pattern [4], [6]. Since manipulating communication networks can introduce runtime overhead, recent works have begun to offload traffic obfuscation onto programmable network switches, which change communication patterns at much higher line rates than CPUs [7]–[9].

Unfortunately, these methods do not directly transfer device fingerprinting in ICS/SCADA environments for two main reasons. First, the leaked features are different. General web traffic obfuscation usually targets the flow level sizes and timing patterns [7] whereas ICS device fingerprinting is carried by the response timing of the protocol exchanges, which stays visible even when the payload is encrypted [10]. Secondly, adding bytes or cover packets is constrained in this setting in a way it is not on the open Internet. A DNP3 link-layer frame carries a declared length and a cyclic redundancy check (CRC) over each block of the frame [11], so bytes inserted at an arbitrary offset change the framing the receiver parses. Making such an insertion valid requires protocol-aware reconstruction

Network and Distributed System Security (NDSS) Symposium 2027
22–26 March 2027, Seoul, Republic of Korea
ISBN 978-1-970672-09-1
https://dx.doi.org/10.14722/ndss.2027.23xxxxx [DRAFT: paper number not assigned]
www.ndss-symposium.org

of the frame and recomputation of its link-layer CRC blocks, which is not what an opaque pad does. The outstation that would otherwise emit cover traffic is a protective relay whose firmware cannot be modified, so a legacy relay cannot simply be made to generate that traffic either. Consequently, a defense in this setting must hide the timing that leaks even after encryption while changing no bytes.

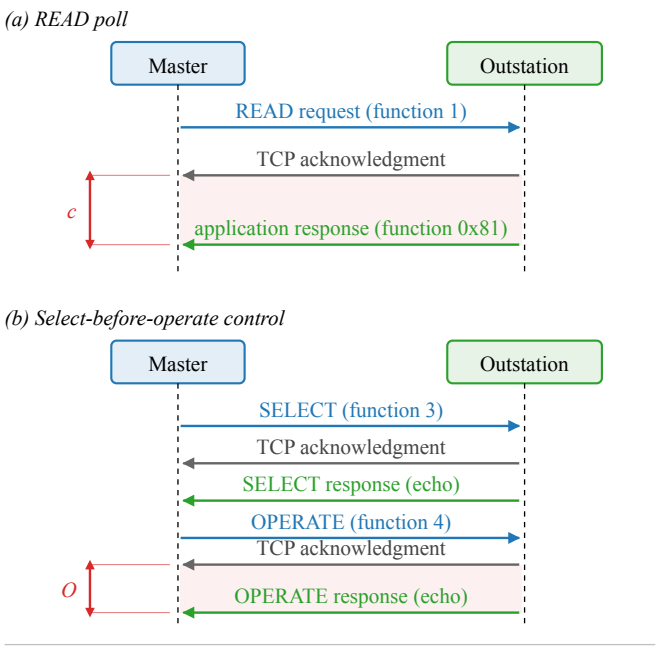
In this paper, we present an in-network mechanism that normalizes an outstation's fingerprintable features using the data plane of the Tofino programmable switch at the outstation edge, without changing the endpoints or modifying the bytes of the DNP3 traffic. This defense holds per transaction responses and releases them at configurable offsets so the master-visible timing then becomes a policy value. The two timing features that Formby et al. [10] use for fingerprinting, the cross-layer response time of a poll and the master-visible timing of a control command, are the two case studies through which we show the framework working; they are not two separate systems. We implement the framework as a P4 program on a single switch, evaluate it on a cyberphysical testbed, and make the following contributions:

- **Design an in-network timing-obfuscation framework** that replaces selected within-transaction timing features with configurable release-policy values, without modifying either endpoint and without changing the protected DNP3 bytes.
- **Implement the framework on an Intel Tofino-1** as a P4 program with a separate acknowledgment-anchored read lane and request-anchored control lane, bounded queues, and fail-open forwarding when a response arrives after its scheduled release.
- **Evaluate the framework against a physical SEL-751A relay** over 22 grouped collection runs, and report policy programmability, timing-feature suppression, the effect on a fixed transaction-class attacker, the leakage that remains against an adaptive attacker, and the latency the mechanism costs.

II. Background and Motivation

In this section we focus on the DNP3 transactions the paper covers, the features that leak from them and how an adversary uses them in fingerprinting, and the opportunity that a programmable data plane gives us to change the timing without touching the endpoints.

The DNP3 read and control transactions. DNP3 carries supervisory control and data acquisition (SCADA) traffic over TCP between a master and an outstation [11]. The master polls the outstation with a READ request (function code 1), and the outstation returns the requested measurements in an application-layer response (function code 0x81). A control command, on the other hand, uses select-before-operate (SBO): the master first sends a SELECT (function code 3) that names an output point, and the outstation responds by echoing the request; the master then sends an OPERATE (function code 4), which the outstation confirms with a response that echoes the command. Because DNP3 runs over TCP, every request is also acknowledged at the transport layer before the application answers. As such, each transaction on the wire is a request, a TCP acknowledgment that carries no DNP3 payload, and a DNP3 response, in that order, as Figure 1 shows. We use these terms consistently in the rest of the paper: the acknowledgment is the TCP segment with the ACK flag that the outstation returns first, and the response is the DNP3 application frame that follows it.



c: acknowledgment-to-response interval of the READ (CLRT).
O: master-visible OPERATE ACK-to-echo interval.

Fig. 1. The DNP3 transactions the paper measures, as seen on the master-facing link; requests in blue, transport-layer acknowledgments in grey, and application responses in green. (a) A READ poll: the request, the outstation's transport-layer acknowledgment, and its application-layer response. The interval *c* between the acknowledgment and the response is the cross-layer response time (CLRT). (b) A select-before-operate control: SELECT and OPERATE each produce an acknowledgment and a response that echoes the command. The interval *O* between the acknowledgment and the echo of the OPERATE is the master-visible OPERATE ACK-to-echo interval.

Why the transaction timing is a fingerprint. The acknowledgment is produced by the outstation's TCP stack as soon as the request arrives, whereas the response is produced by its application only after the device has done the work the request asks for. Consequently, the interval between the two, the CLRT, reflects the outstation's internal processing rather than the network path, and it differs between device models and between the kinds of work they do. Formby et al. used it to tell device types and models apart in a substation network, and used the time a device takes to complete a control operation as a second feature that reveals the physical device behind the command [10]. Our own measurements on a physical SEL-751A relay show the same effect on one device: with no timing defense, the median CLRT is 2.116 ms for a READ, 2.050 ms for a SELECT and 2.937 ms for an OPERATE, each with a long tail, so the three transaction classes are already partly

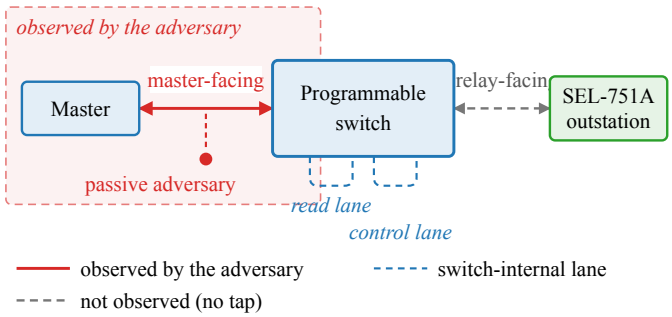


Fig. 2. Observation model. The master reaches the SEL-751A outstation through one programmable switch. The passive adversary sees only the master-facing link (solid red, shaded region); the relay-facing link and the two switch-internal scheduling lanes (dashed) are not observed, by the adversary or by our measurements.

separable from timing alone (Section VI). The function codes remain visible in the plaintext, so the classes themselves are not a secret; what the timing adds is a channel that identifies the device and its operation without any payload, and that survives encryption.

Measured and inferred events. Everything the paper measures is a packet timestamp on the master-facing link, i.e., the request, the acknowledgment, and the response. The switch-internal release times and any event on the relay-facing link are not observed; where the paper names them, they are inferred from the configuration. Physical breaker motion is never measured.

Programmable data planes. A programmable switch exposes a match-action pipeline, written in P4 [12], that parses headers, keeps per-packet state in registers, and places packets into output queues at line rate. Because such a pipeline can delay and release a packet without a host in the loop, it can hold an outstation's acknowledgment and response and release them on a schedule that the operator sets, while changing no endpoint and no byte of the payload.

III. Threat Model and Objectives

Assumptions on the adversary. The adversary is a passive observer on the master-facing link and has the ability to record TCP and IP headers, segment sizes, and arrival times, but neither modifies nor injects packets, as in the fingerprinting attack of Formby et al. [10]. Figure 2 shows the setup. Because DNP3 is usually deployed without transport security across shared substation networks [13], we assume the traffic here is unencrypted. Therefore, the adversary is able to read the exchanges directly, including the DNP3 function codes, and from them it derives the transaction timing of Section II and can train a classifier that separates transaction classes from timing features. The adversary cannot compromise the master or the outstation, control the switch, or read the switch's internal state. It also does not observe the relay-facing link, because in our deployment that link is inside the switch and no tap exists, and it cannot see physical breaker motion. An adversary that actively interacts with the traffic, sends its own probes, or changes the network is out of scope; we stay with the passive observer, which is the setting that prior traffic obfuscation work targets.

Reconnaissance target. The adversary's objective is to fingerprint outstation devices and obtain knowledge to prepare for a later attack. Because control-related attacks such as false data injection attacks work only when the attacker knows the target devices [14], reconnaissance is the enabling step to identify the devices and their weaknesses. The fingerprinting process can succeed without decoding the payloads because it is done using the timing features of the master-outstation exchanges. Following [10], the two timings in scope are the cross-layer response time (CLRT), i.e., the interval between the transport-layer acknowledgment and the application-layer response of a READ or of the SELECT phase of SBO, and the master-visible OPERATE ACK-to-echo interval, i.e., the interval between an OPERATE's acknowledgment and its echo. The CLRT reflects the outstation's internal processing, while the second reflects the physical device behind a control command. The adversary also sees the request-to-acknowledgment interval, which we report as measured.

Two attacker models. A defence that is only evaluated against a classifier retrained on its own output answers the wrong question, so we separate two adversaries. The *fixed* adversary learns the transaction classes from undefended traffic and applies that model unchanged once the defence is deployed; this is the adversary the reconnaissance setting describes, because the profile is built before the defence is met. The *adaptive* adversary collects obfuscated traffic and retrains on it, which is a strictly stronger assumption and bounds what remains learnable. We report both.

Research objectives. Our objective is therefore to remove these features from the master-facing observing adversary. We state it as five properties that the evaluation answers one by one.

- **RO1 (read timing).** The visible CLRT of READ and of the SELECT phase of SBO is a configurable policy value rather than the outstation's own processing signature.
- **RO2 (control timing).** The master-visible OPERATE ACK-to-echo interval stays at a policy value while the relay-facing hold is drawn from a configured codebook.
- **RO3 (timing leakage).** The timing features carry less information about the transaction class, and a fixed adversary is reduced to chance.
- **RO4 (budget, coverage, and cost).** The release budget, the fraction of traffic it can cover, the residual it leaves, and the latency it costs are characterised rather than assumed.
- **RO5 (protocol preservation).** The external DNP3 workload still completes: every request is answered and every control operation returns a success status.

What is out of scope. The framework does not hide the function codes, which stay visible in plaintext DNP3; RO3 is about the timing channel, not the command semantics. It does not change packet sizes, and no size-related claim is made in this paper. Additionally, it does not claim that two

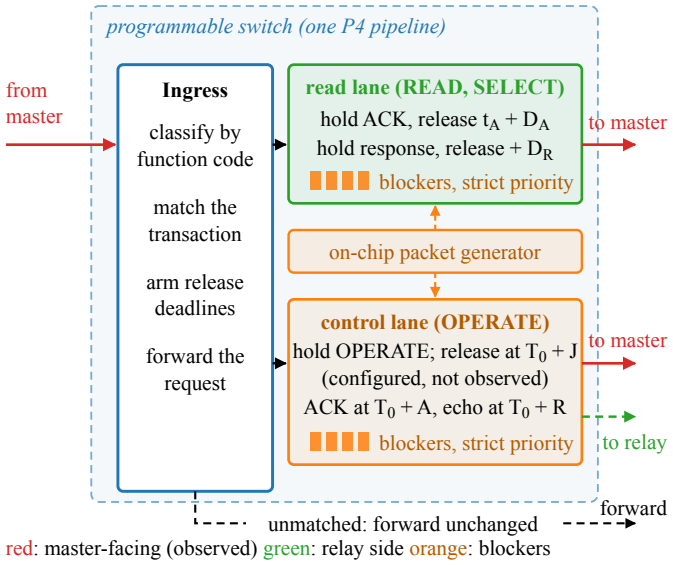


Fig. 3. Design of the hold and timing obfuscation mechanism. Ingress classifies each DNP3 transaction and arms its release deadlines. The read lane holds the acknowledgment and the response of a READ or SELECT and releases them relative to the outstation's acknowledgment. The control lane holds an OPERATE, releases it toward the relay at a configured delay that is not observed, and releases its acknowledgment and echo relative to the request. The generator fills the blocker reservoirs (orange); a strict-priority scheduler drains them first, which realizes an armed release deadline when the matching packet arrives within the schedulable window, and otherwise forwards it under the bounded fail-open path so availability is preserved. Red arrows are master-facing and observed; the green dashed arrow is the configured relay-facing release at $T_0 + J$, not observed. Unmatched traffic is forwarded unchanged.

different devices become indistinguishable: the evaluation has one outstation, and what it shows is that this outstation's timing signature is replaced by a policy signature for the exchanges the budget can cover. The relay-facing link is not instrumented, so the per-transaction hold and the release toward the outstation are configured rather than observed.

IV. Design

In Figure 3, we show the design of the hold and timing obfuscation mechanism. When a request from the master enters the switch, the ingress classifies it by its function code, records the state it needs to recognize the outstation's answer, and forwards the request through the pipeline without a delay. When the outstation answers, the answer goes through the same pipeline, but this time the program holds the acknowledgment and the response in switch queues in the traffic manager and releases each one at a predetermined offset. In this way, the master sees a timing value that the policy sets instead of the timing that the outstation itself produces, which is the signature used for fingerprinting. Because the read and the control responses must not interfere with each other, we implement a separate hold lane for each, and because the two transactions expose different information, each lane uses its own anchor for the offsets. The rest of the section presents the mechanisms and what they require.

Classification and transaction state. The switch parses the DNP3 link and application headers of every packet on the protected session and classifies each packet as a request (READ, SELECT, or OPERATE, by function code), an acknowledgment (the outstation's first segment with the TCP ACK flag after a request), a response (a DNP3 application frame from the outstation), or other. A small set of registers holds the state of the one transaction in flight on the session, i.e., an active-transaction tag, the expected acknowledgment number of the outstation's reply, the armed deadlines, and, for control, the identity of the held OPERATE. The state is written once per event and read by the packets that follow, so a duplicate acknowledgment or a retransmitted request cannot arm a second deadline.

Anchoring the read deadlines to the acknowledgment. A held packet must leave the switch at a specified deadline, but the switch and P4 have no general-purpose timer. For a READ or a SELECT, the feature to remove is the CLRT, the interval between the acknowledgment and the response. Therefore, when the outstation's acknowledgment reaches the switch at time t_A, the program arms two deadlines from the policy offsets D_A and D_R,

$$t_{\text{ack}} = t_A + D_A, \qquad t_{\text{resp}} = t_A + D_A + D_R, \tag{1}$$

at which it releases the TCP acknowledgment and the DNP3 response to the master respectively. Consequently, the CLRT interval the adversary measures is

$$\text{CLRT} = t_{\text{resp}} - t_{\text{ack}} = D_R, \tag{2}$$

a policy value that replaces the CLRT interval that the outstation's internal processing produces and that serves as its fingerprint. Because the switch can delay a packet but cannot release it before it exists, equation (2) holds only when the response has arrived by $t_A + D_A + D_R$; as such, the sum $D = D_A + D_R$ is a release budget: it sets which exchanges the switch can place on the schedule, and it is not the delay added to every exchange. Exchanges the outstation answers within the budget are pinned; the rest arrive after their scheduled release and are forwarded when they arrive, which Section VI-B quantifies. A second bound applies from above, because the reservoir that starves the queue carries a finite pass budget and cannot hold a packet past the fail-open horizon H. Note that the request-to-acknowledgment interval then becomes the outstation's own acknowledgment latency plus D_A: it is shifted by a constant, but the sub-millisecond spread of the outstation's TCP stack remains in it.

Anchoring the control deadlines to the request. A control command is different, because the framework also holds the OPERATE itself before it reaches the relay. This separates the moment the master sent the command from the moment the relay acts on it, which an operator may want for a control command; but the outstation's acknowledgment then exists only after the release, so a deadline of the form $t_A + D$, as in (1), would carry the length of the hold into the observable. Therefore, the control path is anchored to the request instead. When the OPERATE arrives at time T_0, the switch holds it

and releases it to the relay at an internal delay J, while the acknowledgment and the echo the master sees are placed at fixed offsets from T_0,

$$t_{\text{ack}} = T_0 + A, \qquad t_{\text{echo}} = T_0 + R, \tag{3}$$

where A and R are configured such that A exceeds J plus the outstation's acknowledgment latency, which the control plane checks before it installs the values. The master-visible OPERATE ACK-to-echo interval is therefore

$$O = t_{\text{echo}} - t_{\text{ack}} = R - A, \tag{4}$$

in which J does not appear. J is drawn per transaction from a configured set, so the relay-facing release time varies while the master-facing observable does not. In our experiments $D_R = R - A = 4$ ms, so both case studies present the same 4 ms policy value to the observer.

A family of release policies. The two offsets are independent, and the choice of which of them is non-zero names a policy. Setting $D_A = 0$ releases the acknowledgment at once and holds only the response, a response-only policy. Setting $D_R = 0$ holds the acknowledgment and lets the response follow it immediately, an acknowledgment-only policy. Setting both above zero holds each packet to its own deadline, which is the dual-deadline policy we implement and evaluate. A fourth policy would release the held acknowledgment on an event, e.g., the arrival of the matching response, rather than on a deadline; it is driven by an event and is therefore not a parameter case of the other three, and we do not implement it. Because the response-only and acknowledgment-only policies are limits of one mechanism, a single datapath covers the family, and an operator selects a point in it by choosing the two offsets rather than by loading a different program.

Choosing the budget. The switch can release only a packet it is already holding, so the budget $D = D_A + D_R$ has to exceed the outstation's own response time. Consequently D sets which exchanges can be placed on the schedule and bounds the delay the framework adds, and choosing it is a trade-off rather than a free parameter; the delay actually added to an exchange depends on how long the outstation itself took and is therefore at most D, not equal to it. The observer, on the other hand, sees only D_R, because by (2) the visible interval is the gap between the two releases. Since the budget and the observable are set by different quantities, an operator can hold the cost fixed and still choose what the timing feature looks like. A second bound comes from the other direction: the reservoir that starves the queue carries a finite pass budget, so a hold cannot outlast the fail-open horizon H after which the packet is released regardless. The usable region is therefore D above the outstation's response-time tail and below H, and Section VI-B measures both edges.

Security and deadlines. The anchor matters as much as the offsets. On the read path the request is forwarded at once and only the reply is held, so anchoring to the outstation's acknowledgment costs nothing and lets the hold start as late as possible; by (2), the CLRT becomes D_R regardless of how long the outstation took. On the control path the command is held, so the acknowledgment is a function of J, and only a request-anchored rule keeps J out of the observable, by (4). As a result, an adversary that measures either interval reads a policy value rather than the device's own processing time for that interval, and because J is not present in (4), the adversary cannot subtract a known offset to recover the device's original processing time either. This replaces the selected timing feature and nothing more. The DNP3 function codes stay in plaintext, the direction of each packet stays visible, the request-to-acknowledgment interval still carries the outstation's own sub-millisecond acknowledgment latency shifted by a constant, and the read and control paths keep different anchors, so other features remain available to an observer.

Reservoir tokens as data-plane timers. As the programmable switch has no timer, to achieve a deadline from scheduled packets alone, a held packet must wait in its queue behind a reservoir of blocker packets that sit in a higher-priority queue on the same port. A strict-priority scheduler in the traffic manager drains the blockers first, so the held packet cannot leave until the reservoir is empty. If τ is the time to drain one blocker, a reservoir of K blockers will hold a packet for about $K\tau$, and the control plane sizes K for each deadline. The blockers are generated by the switch's on-chip packet generator when the request arrives, they circulate on internal ports, and they are dropped in the switch when they expire. Consequently, they never leave the switch and are not visible to the master nor the outstation. A tag in each blocker names the transaction and the lane it belongs to, so a stale blocker from an earlier transaction cannot delay a later one.

Independent scheduling lanes. A read response and a control response can be held at the same time, and if they were serialized in a single lane, whichever holds longer would delay the other and hence reshape the timing policy that is fixed. This therefore gives each treatment its own internal recirculation lane and its own traffic manager queues, which are independent of each other, and the register state of the two lanes is likewise separate.

Unmatched transactions and failing open. A defense on a control path must not become an availability risk. As such, a packet that does not belong to a protected session, a request the framework does not recognize, an acknowledgment with no armed transaction, or a response that arrives after its transaction has retired is forwarded unchanged. In this way, an unexpected exchange loses its protection but not its delivery. Every hold is also bounded: a fail-open budget on the number of blocker passes releases a held packet if its reservoir has not drained within a configured horizon and clears the transaction state, so a lost acknowledgment or response cannot leave a deadline armed forever. With the timing mode switched off, the same program forwards every packet immediately and arms nothing.

Byte preservation. The framework only holds and forwards packets. Since it neither pads a response, injects a packet into the protected exchange, nor edits a DNP3 field, every byte the master reassembles is the byte the relay produced, and the

payload's cyclic redundancy checks remain valid.

V. Implementation

Target and program. We implement the design as a single $P4_{16}$ program for the Tofino Native Architecture on an Intel Tofino-1 switch, compiled with the Barefoot SDE 9.13 compiler. The program occupies all twelve ingress match-action stages of one pipeline and six egress stages, with 112 tables. The classification of Section IV is a parser for the Ethernet, IP, TCP, and DNP3 link and application headers followed by a chain of match-action tables, and the transaction state is a set of one-entry registers accessed by stateful ALU actions, one access per packet per register. The final forwarding decision is made by one table keyed on a compact outcome computed by the earlier stages, so that every disposition (forward, hold, release, or drop an expired blocker) is an explicit entry.

Ports and lanes. The master is cabled to one front-panel port and the relay to another; these are the only external links. Two further ports are configured in loopback and carry no cable: one is the read lane and the other the control lane of Section IV, each with its own hold and blocker queues at descending strict priority.

Timing state. The deadlines are 32-bit nanosecond timestamp words quantised to a 256 ns grid, taken from the ingress timestamp of the anchoring packet. On the read path the acknowledgment's arrival is the anchor and the offsets D_A and $D_A + D_R$ are added to it. On the control path the OPERATE's arrival is the anchor and A, R, and J are added to it, and the acknowledgment and echo deadlines are written when the OPERATE is admitted, so that the outstation's later acknowledgment cannot re-anchor them. J is selected per transaction by a table keyed on a data-plane random value from a configured set of admissible holds. The offsets D_A, D_R, A, R, the J set, the fail-open budget, and the timing mode (off or on) are all runtime table entries.

Control plane. A Python control plane over the switch's runtime interface brings up the ports, creates the queues, installs the multicast and generator configuration, writes the timing parameters, and reads every value back. The values are checked before they are installed: the offsets must be multiples of the 256 ns quantisation step, A must exceed the largest J plus the outstation's acknowledgment latency, and R must exceed A. A failed check leaves the mechanism disabled rather than partly configured. The guarded test harness that issues a physical SELECT and OPERATE permits only two isolated control points on the relay and refuses the breaker-close point.

Which policies the build realizes. The program defines the release policies of Section IV as mode values, but the disposition tables that seed the blocker reservoir carry entries only for the bypass path and for the dual-deadline policy. Because a mode with no entry never seeds a reservoir, it never arms a hold, so the loaded build realizes those two and no other, even though the remaining mode values stay installable from the control plane. This is a consequence of fitting the pipeline: collapsing the disposition logic into two priority-ordered ternary tables is what brought the program within the twelve ingress stages, and the separate gates for the other policies did not survive that collapse. We therefore evaluate the dual-deadline policy, and the response-only and acknowledgment-only policies appear in Section VI-B as limits of its two offsets rather than as run-time selections.

What ran, exactly. The evidence in Section VI was produced by one program loaded once, whose source and compiled binary are identified by hash in the repository. That program is a combined implementation: besides the timing mechanism described here, it carries a size-shaping datapath that we developed separately and that can be enabled independently of the timing mode. For the campaign reported in this paper the size datapath is disabled in both arms, which we verified by control-plane readback and on the wire, where every response arrives as one application-layer segment and every capture carries the same frame and byte counts in both arms. The two arms therefore differ only in the release schedule. We name them *Timing OFF* and *Obfuscated* rather than native and defended, because the Timing OFF arm is the same binary with the release schedule bypassed and not an untouched relay. This paper evaluates timing only; the size datapath is not evaluated, claimed, or figured.

VI. Evaluation

We answer the five objectives of Section III on a physical testbed. Each subsection states the question, the method, the result, and what the result does not settle.

A. Testbed and Dataset

Testbed. A server running a DNP3 master reaches a physical SEL-751A feeder relay through an Intel Tofino-1 switch running the program of Section V. The master has no other path to the relay, which we confirmed by disabling the relay-facing port and observing that the relay became unreachable. All timestamps are taken on the master-facing link, so every interval reported here is one a passive observer on the control network could record.

Arms. *Timing OFF* forwards the outstation's acknowledgment and response as they arrive. *Obfuscated* holds them and releases them on the deadlines of Section IV. Both arms run the same loaded program with the size carve disabled, so the only difference between them is the release schedule.

Units. We report per DNP3 *exchange*: one request, the transport acknowledgment that follows it, and the application response. A capture contains 480 exchanges. The same capture contains 440 high-level operations if a SELECT and its OPERATE are counted as one select-before-operate operation, and one TCP connection. We use exchanges throughout and never mix the two conventions.

Corpus. The campaign is 22 grouped collection runs in one approximately five-hour window, six captures per run, 132 captures in total. It contains 63,360 exchanges, 31,680 per arm, of which 26,400 are READ, 2,640 SELECT and 2,640 OPERATE in each arm. A separate 19-point parameter sweep, 5,860 further exchanges, was collected on the same testbed to characterise the release policy itself.

Validation. We re-derived every exchange from the raw captures with a reader independent of the one used during collection, and compared the result against the frozen transaction table row by row on identity, ordering and every stored interval. All 132 captures agree: no retransmission, no malformed frame, no unpaired request, no out-of-order timestamp, and a well-formed response to every request. Each capture holds 1,448 frames and 130,708 captured bytes. The captures are equal in size but not identical; all 132 hashes differ, and all 22 per-run manifests verify.

B. RO4a: Policy Programmability and the Operating Envelope

Question. Is the interval a passive observer records actually a policy value the operator sets, and over what range of settings does the mechanism hold?

Method. We installed 19 release policies on the switch in turn and ran a fixed DNP3 workload through each. Two series answer the question. The first holds the total budget at $D = D_A + D_R = 24$ ms and moves the split between the two offsets, which changes the visible CLRT while leaving the total unchanged. The second fixes D_R at 4 ms and ramps D_A upward until the mechanism leaves its operating region. Each point is summarised by the median over its READ exchanges.

Result. In Figure 4(b), the measured CLRT follows the configured D_R along the identity line across the whole series: targets of 1, 2, 4, 8, 12, 16, 20 and 22 ms produce measured medians of 0.998, 1.999, 3.999, 8.001, 12.001, 16.000, 20.001 and 22.001 ms, while the end-to-end response time stays between 25.30 and 25.34 ms at every point. The leaking interval and the cost of the exchange are therefore set independently. In Figure 4(c), the request-to-acknowledgment interval tracks D_A up to 30 ms, where the measured median is 30.571 ms and the CLRT still sits at 4.001 ms. Beyond that the acknowledgment interval saturates at about 31.07 ms and the CLRT can no longer be held: it rises to 5.503, 7.498 and 9.507 ms at D_A of 32, 34 and 36 ms. In Figure 4(a), the budget of 24 ms covers 99.900% of Timing OFF read-lane exchanges; 29 of 29,040 arrive too late to be held, 28 of them READ and one SELECT.

Interpretation and boundary. The observable is a policy value over a usable range, which is what makes the mechanism a framework rather than one fixed setting. The range is closed on both sides: from below by the outstation's own response time, which the switch cannot anticipate, and from above by the finite pass budget of the reservoir that starves the queue. The saturation point sits close to the fail-open horizon H computed in the control plane, but H is a control-plane quantity rather than a bound the data plane enforces, so we report the observed transition and not a proof that the two coincide. The offsets for each sweep point are read from the archived configuration table; no per-point control-plane readback exists, so configuration provenance for the sweep is partial.

C. RO1: READ and SELECT Timing

Question. Does the read-path rule of (1) replace the outstation's own CLRT with the policy value?

Method. We compare the CLRT of READ and of the SELECT phase of SBO between the two arms over all 22 grouped runs, as full empirical distributions rather than summary statistics alone, because the shape of the Timing OFF distribution is what makes a single observation informative.

Result. In Figure 5(d), the median CLRT under Timing OFF is 2.116 ms for READ and 2.050 ms for SELECT, with interquartile ranges of 2.778 ms and 2.697 ms. Under the mechanism both medians are 4.000 ms and the interquartile range of each falls to 0.006 ms, a reduction of more than two orders of magnitude in spread. Figure 5(a, b) shows why the median alone understates the change: the Timing OFF distributions rise in a small number of discrete steps, so a single observation is informative about the class, whereas the obfuscated distributions are a step at the scheduled release.

Interpretation and boundary. The release instant is computed from the outstation's acknowledgment and a fixed offset, so it no longer depends on how long the device took. The concentration is not total. A tail remains: the largest obfuscated READ interval is 57.182 ms, and 24 of 26,400 READ exchanges depart from the scheduled release by more than 1 ms. These are exchanges whose response reached the switch after its scheduled release, which is the coverage boundary of Section VI-B seen from the other side.

Stability within the campaign. Figure 6 plots each grouped run in acquisition order. The obfuscated median sits at the scheduled release in every run and for every class, while the Timing OFF medians stay separated, so the effect is not an artefact of one run or of drift over the five hours. These runs share a testbed, a device, and one session of the same afternoon, so this is stability within the campaign and not evidence of generalisation across days, devices, or deployments.

D. RO2: Master-Visible OPERATE Interval

Question. Does the control-path rule of (3) hold the master-visible ACK-to-echo interval at the policy value?

Method. The control path is anchored to the request, so its observable is $O = R - A$ and the read-path budget D is not a schedulability criterion for it. We therefore report OPERATE on its own terms and never place it in the read-lane coverage denominator. The switch draws the internal hold J per transaction from the configured codebook $\{2, 6, 12\}$ ms.

Result. The OPERATE median moves from 2.937 ms under Timing OFF to 4.000 ms under the mechanism, with the interquartile range falling from 2.827 ms to 0.006 ms (Figure 5(c, d)). Under the configured $\{2, 6, 12\}$ ms codebook, the master-visible OPERATE ACK-to-echo interval remained concentrated near the configured 4 ms policy value across all 22 grouped runs.

Interpretation and boundary. The measurement is master-facing only. The campaign records the configured codebook but not the realized per-transaction draw, so we do not report a result per J, and we do not claim that the interval was shown to be insensitive to the codebook. The relay-facing release at T_0+J, the number of OPERATE frames that reached the relay,

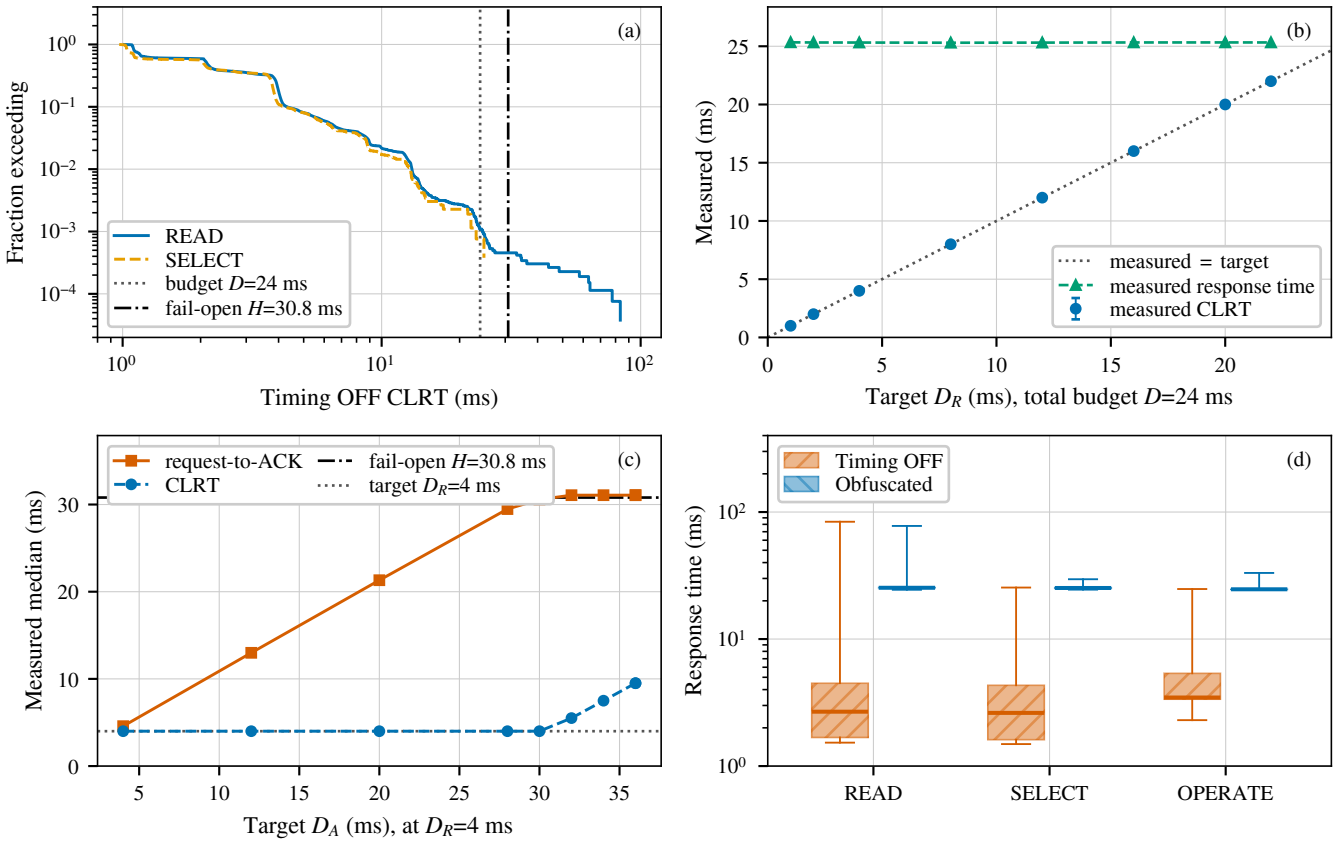


Fig. 4. **The release policy is programmable, and its budget is bounded on both sides.** (a) Fraction of Timing OFF read-lane exchanges, READ and the SELECT phase of SBO only, whose CLRT exceeds a given value, with the release budget D and the fail-open horizon H. (b) Measured hardware sweep at a fixed total budget $D = 24$ ms: the visible CLRT follows the configured D_R along the identity line while the end-to-end response time stays put. (c) Measured sweep of D_A at fixed D_R: the request-to-acknowledgment interval tracks the target until it saturates near H, beyond which the CLRT can no longer be held at its target. (d) End-to-end response time per class. Bars in (b) span the interquartile range; boxes in (d) span the quartiles with whiskers over the full support.

and any physical actuation are outside what was observed. What the evidence supports is a statement about the interval the master sees.

E. RO3: Transaction-Class Leakage

Question. How much does the timing tell an observer about the transaction class, and does that depend on whether the observer can retrain?

Method. We treat this as a three-class problem over READ, SELECT and OPERATE, for which chance balanced accuracy is one third. Features are the two independent intervals, request to acknowledgment and acknowledgment to response; the total response time is their sum and carries no independent information, so it is excluded. Mutual information is estimated for the CLRT and converted to bits, and is compared against a null built by shuffling class labels within each grouped run over 1,000 permutations, which preserves the per-run class counts; we report the empirical Monte Carlo p-value and its resolution. Every fold leaves out one grouped run. We report the spread of the 22 held-out-run scores descriptively, because those folds share training data and are not independent, so a bootstrap over them would not be a confidence interval.

Result. In Figure 7, the fixed adversary trained on Timing OFF traffic reaches 0.651 balanced accuracy on held-out Timing OFF exchanges using the CLRT alone, and 0.733 using both intervals. Applied unchanged to obfuscated traffic it falls to 0.333 and 0.334, which is chance. Mutual information between the CLRT and the class falls from 0.383 bits to 0.004 bits; the Timing OFF value lies far above its permutation null at the resolution of the test ($p = 0.001$ with 1,000 permutations), and the obfuscated value lies inside its null ($p = 0.096$). The adaptive adversary, retrained on obfuscated traffic, remains at chance on the CLRT alone but recovers to 0.651 when the acknowledgment latency is added.

Interpretation and boundary. For the evaluated fixed Random-Forest attacker and this feature set, three-class transaction identification falls to chance, and the cross-layer response time stops carrying information about the class. That is the adversary the threat model describes, because the profile is built before the defence is met. It is not every adversary and not every classifier: these numbers characterise the attacker we

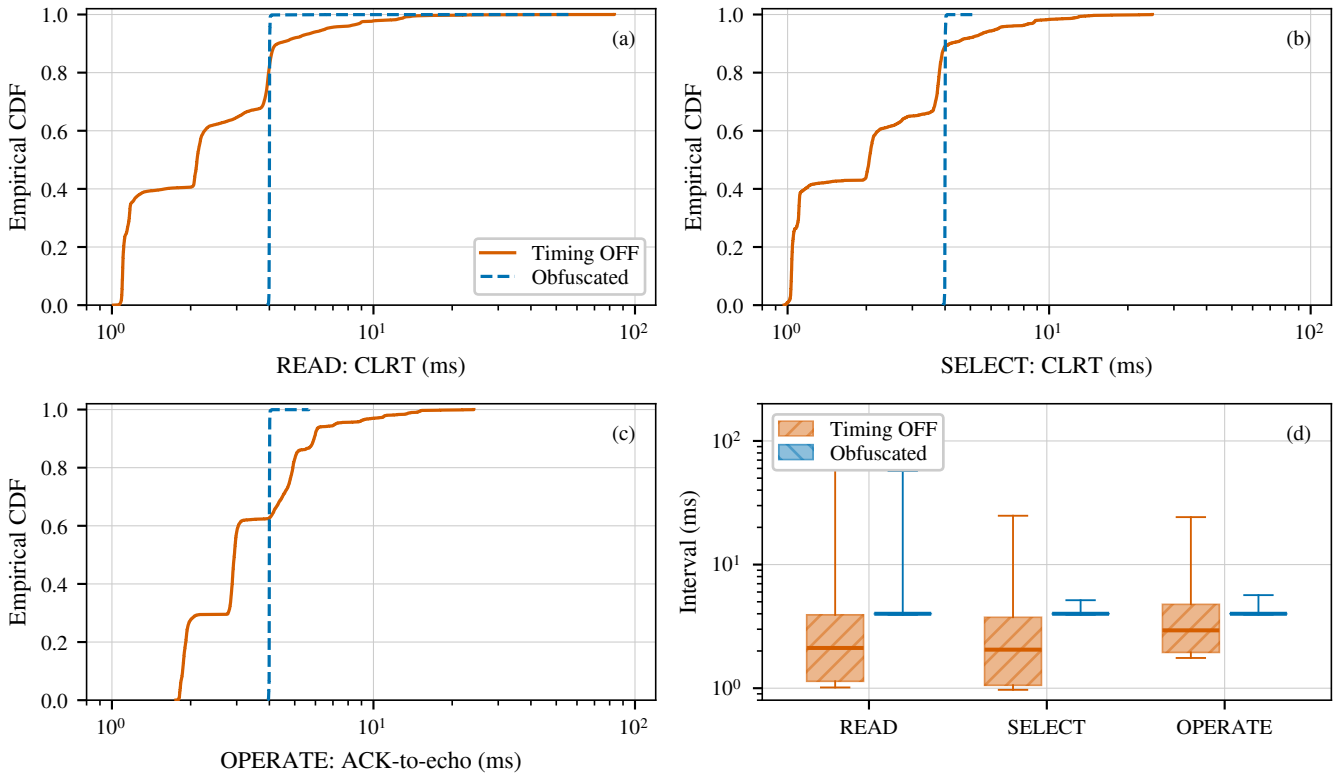


Fig. 5. **Measured interval per transaction class, over 22 grouped runs.** (a) READ and (b) the SELECT phase of SBO report the cross-layer response time; (c) OPERATE reports the master-visible ACK-to-echo interval, which is a different anchor and not a complete SBO transaction. The abscissa is logarithmic and spans the full support, so no observation is clipped. (d) The same measurements as quartiles, with whiskers spanning the full support.

evaluated and do not generalise to all fingerprinting classifiers. The mechanism also does not remove every timing feature. The read and control paths are anchored differently, so their acknowledgment latencies differ, and an adversary willing to collect obfuscated traffic and retrain can exploit that difference. The confusion panels show its shape: OPERATE becomes separable while SELECT stays confused with READ. Closing this residual requires the two paths to share an anchor, which we have not implemented.

F. RO5: Cost, Protocol Preservation, and Safety

Question. What does the mechanism cost on the observed link, and does the DNP3 workload still complete?

Result. In Figure 4(d), the median response time rises by 22.7 ms for READ, 22.6 ms for SELECT and 21.2 ms for OPERATE. The added latency is close to, but not equal to, the release budget, because it depends on how long the outstation itself took. On the master-facing link each capture carries 1,448 frames and 130,708 captured bytes for 480 exchanges in both arms, that is 3.017 frames and 272.3 bytes per exchange, identical in the two arms: the mechanism moves packets in time without adding a frame or a byte to that link. Across all 63,360 exchanges every request was answered with a well-formed response, and all 5,280 SELECT and OPERATE exchanges returned a success status.

Interpretation and boundary. The workload completes unchanged, which is a precondition for deploying the mechanism in front of a live outstation. The overhead statement is about the observed link: the switch also generates internal blocker traffic that drives the release schedule, and that traffic never appears on the master-facing link and is not counted here. The added latency is a real operational cost that an operator must weigh against the polling period and any protocol timeout.

G. Limitations

The evidence is one SEL-751A behind one Tofino-1, in one approximately five-hour campaign of 22 grouped runs on a single testbed, so it does not establish behaviour across days, devices, or deployments; the 22 runs are grouped collections and not independent deployments. DNP3 function codes remain visible in plaintext, so the result concerns the timing channel and not the concealment of operations. The classification we evaluate is transaction-class classification, not device-model identification: with one outstation, nothing here shows that two devices become indistinguishable. Only the master-facing link was captured, so the realized per-transaction hold J, the relay-facing release, and any physical actuation are configured rather than observed, and nothing here demonstrates that exactly one OPERATE reached the relay. A fixed read-path release budget leaves a measurable

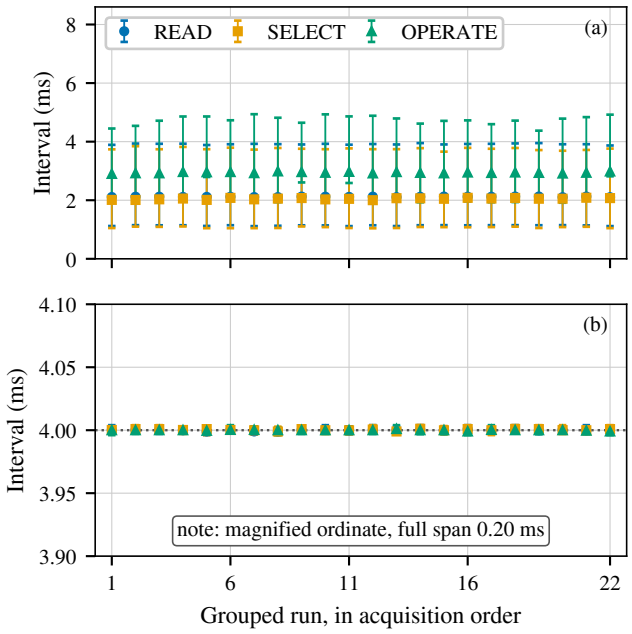


Fig. 6. **Within-campaign stability across the 22 grouped runs**, in acquisition order. (a) Timing OFF and (b) Obfuscated. Markers are the run median and bars span the interquartile range. *Panel (b) uses a magnified ordinate spanning only 0.20 ms around the scheduled release.* The 22 runs come from one approximately five-hour campaign on one relay behind one switch and are not independent replications.

late tail rather than eliminating it, which the fail-open path bounds. An adaptive adversary that retrains on obfuscated traffic recovers part of the class signal from the difference between the two anchors, and our attacker results characterise the Random-Forest attacker we evaluated rather than every classifier. Configuration provenance is partial: the campaign parameters are corroborated by the wire and by the archived control-plane readback, and the sweep offsets are read from the archived configuration table without a per-point readback. Size obfuscation is outside this paper: the mechanism changes no packet size, and we make no size, padding, splitting or segmentation claim.

VII. Related Work

Device fingerprinting. Devices can be identified from what they emit, even without any access to the payload. Kohno et al. fingerprint a remote host from the skew of its clock as seen in TCP timestamps [15], and GTID fingerprints a device and its type from the timing signature of its packets [16]. These works define the threat we address; our objective, on the other hand, is to remove the timing features they rely on from the master-facing observation.

ICS and power-system fingerprinting. Formby et al. brought fingerprinting to control systems with the cross-layer response time and the physical operation time, measured at the device, of DNP3 outstations [10], and Gu et al. added device physics as a feature [17]. Jeon et al. fingerprint SCADA devices passively without deep packet inspection [18]. The reconnaissance these techniques serve enables false-data injection and process-control attacks [14], [19]. Defenses in this setting have so far worked at the content and connectivity layer: DefRec virtualizes physical functions to present deceptive content [20], RAINCOAT randomizes data acquisition and injects decoy measurements [21], and a companion testbed evaluates such anti-reconnaissance approaches on grid infrastructure [22]. Compared to these, our framework works below the semantics they reshape and complements them: it changes when the wire carries the outstation's answer, not what the answer says.

General traffic obfuscation. Traffic morphing transforms one class's packet-size distribution into another's [3], while Dyer et al. show that per-packet padding and fragmentation still leave exploitable features [4]. Tamaraw fixes packet size and rate at a bandwidth cost [6], and WTF-PAD pads inter-packet gaps adaptively [23]. However, deep fingerprinting attacks defeat several of these [5], and their accuracy at Internet scale is contested [24]. The same size-and-timing channel identifies smart-home devices under encryption, where shaping is the countermeasure [25], and Pacer and NetShaper shape a tenant's traffic to a schedule that bounds side-channel leakage in the cloud [26], [27]. These defenses assume an encrypted payload and a cooperating end host or hypervisor that can add cover traffic. Our framework, in contrast, targets a plaintext protocol whose feature is one interval inside a request-response exchange, adds no byte, and runs where the outstation cannot be changed.

Programmable data-plane traffic transformation. P4 defines the programmable parser and match-action model we use [12]. PIFO defines programmable scheduling at line rate [28], and SP-PIFO approximates it with strict-priority queues on today's switches [29], which is the primitive our reservoirs use. Ditto pads and injects chaff to a fixed pattern at line rate [7], Minos morphs and schedules encrypted traffic on a programmable switch with low overhead [8], and Securitas mixes fragmentation and insertion under a learned policy across several data planes [9]. NetHide obfuscates topology in the data plane against link-flooding reconnaissance [30]. Closest to our setting, Hu et al. use programmable switches to enhance industrial-protocol security [31]. Our framework differs from these in the observable it addresses, i.e., the master-visible timing of a DNP3 transaction, and in holding packets rather than adding or reshaping them.

DNP3 timing and security. East et al. catalogue attacks on DNP3 and the absence of encryption that makes its semantics visible [13], and specification-based and state-based intrusion detection has been adapted to DNP3 [32], [33]. These do not change the timing an observer sees. The physical operation-time feature we address on the control path is the one Formby et al. measured on a physical outstation [10]; we report what the master sees under the defense and, unlike a measurement of the device, we do not observe the relay-facing side.

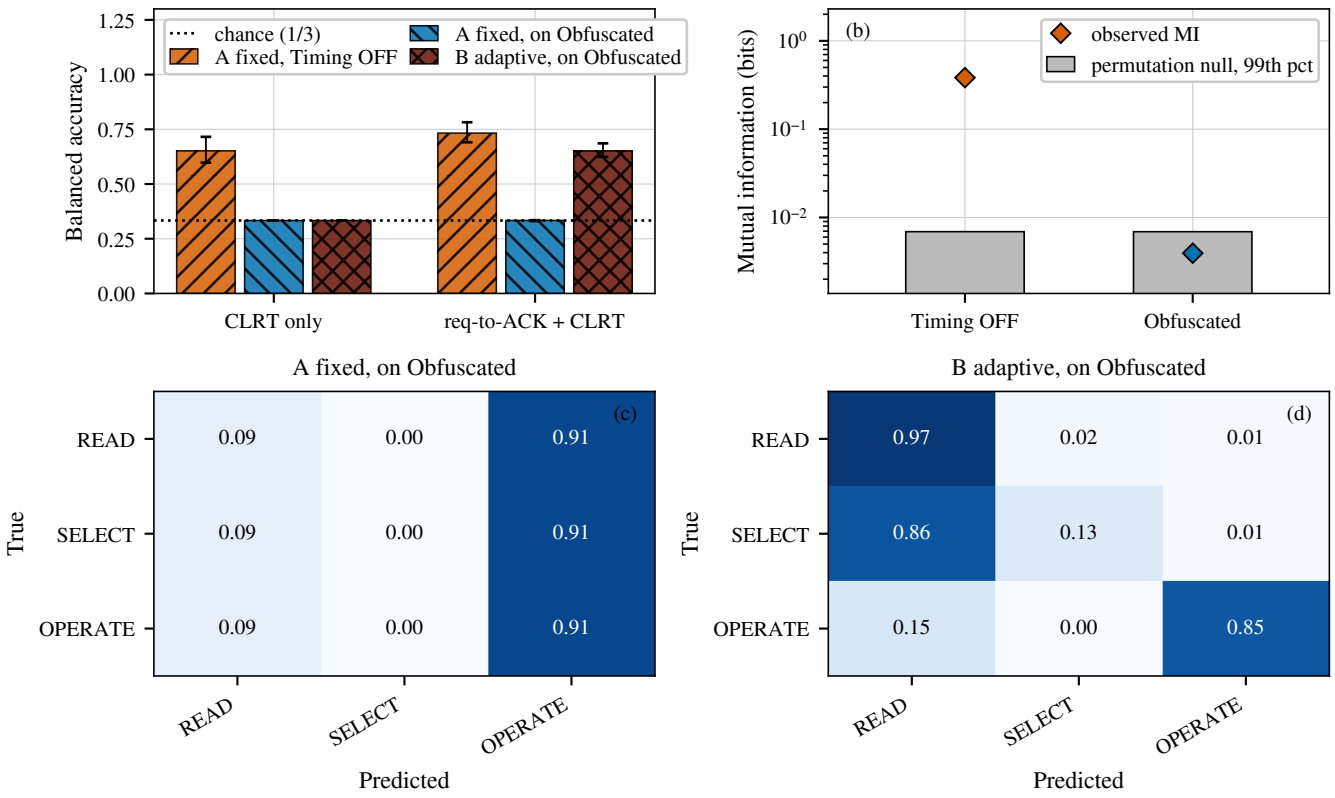


Fig. 7. **Transaction-class leakage under the two attacker models**, leaving out one grouped run at a time over all 22 runs. (a) Balanced accuracy of the evaluated Random-Forest attacker; bars are the mean over the 22 held-out runs and the whiskers span the full range across those runs, which is within-campaign variability and not a confidence interval, because the folds share training data. (b) Observed mutual information between the CLRT and the transaction class against the 99th percentile of a within-run permutation null. (c) and (d) Row-normalised confusion over all 22 held-out runs, using both intervals.

VIII. Conclusion

Response timing lets a passive observer on a DNP3 control network learn what an outstation is doing, and the endpoints that leak it cannot be modified. We presented a timing-obfuscation framework that runs in the data plane of one programmable switch, holds the packets that carry an outstation's timing, and releases them on operator-set offsets, with one rule anchored to the outstation's acknowledgment for polls and the SELECT phase of control, and another anchored to the request for OPERATE. Implemented as a P4 program on an Intel Tofino-1 and evaluated against a physical SEL-751A relay over 63,360 DNP3 exchanges, it moved the cross-layer response time to the configured release value and reduced its interquartile range from about 2.8 ms to 0.006 ms, and a measured sweep on the switch showed the observable following the configured offset across a usable range while the cost of the exchange stayed fixed. For the fixed Random-Forest attacker we evaluated, one that profiles the device beforehand and applies that model unchanged, three-class transaction identification fell from 0.651 balanced accuracy to approximately chance. The result is bounded: it covers the exchanges the release budget can schedule, it leaves the acknowledgment latency of the two paths distinguishable to an adversary willing to retrain, it characterises the attacker we evaluated rather than every classifier, and it concerns one relay behind one switch over one campaign. Extending the evaluation to several outstation models and to campaigns spanning days would show whether a single release policy holds across devices and over time.

References

[1] R. M. Lee, M. J. Assante, and T. Conway, "Analysis of the Cyber Attack on the Ukrainian Power Grid," Electricity Information Sharing and Analysis Center (E-ISAC) and SANS Institute, Defense Use Case, Mar. 2016.

[2] R. Langner, "Stuxnet: Dissecting a Cyberwarfare Weapon," *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.

[3] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic Morphing: An Efficient Defense Against Statistical Traffic Analysis," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2009.

[4] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-Boo, I Still See You: Why Efficient Traffic Analysis Countermeasures Fail," in *2012 IEEE Symposium on Security and Privacy (S&P)*, 2012.

[5] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018.

[6] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A Systematic Approach to Developing and Evaluating Website Fingerprinting Defenses," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2014, pp. 227–238.

[7] R. Meier, V. Lenders, and L. Vanbever, "Ditto: WAN Traffic Obfuscation at Line Rate," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022.

[8] Z. Wang, Q. Li, G. Xie, D. Zhao, K. Li, Z. Fan, L. Ma, and Y. Jiang, "Minos: A Lightweight and Dynamic Defense against Traffic Analysis in Programmable Data Planes," in *Proceedings of the 2025 USENIX Annual Technical Conference (ATC)*, 2025.

[9] G. Xie, Q. Li, Z. Shi, G. Antichi, Y. Zhu, K. Li, C. Weng, S. Miano, Y. Jiang, and M. Xu, "Defending against Traffic Analysis Attacks with Flexible In-Network Obfuscation," in *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026, pp. 2043–2063.

[10] D. Formby, P. Srinivasan, A. Leonard, J. Rogers, and R. Beyah, "Who's in Control of Your Control System? Device Fingerprinting for Cyber-Physical Systems," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2016.

[11] IEEE, "IEEE Standard for Electric Power Systems Communications—Distributed Network Protocol (DNP3)," 2012.

[12] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming Protocol-Independent Packet Processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[13] S. East, J. Butts, M. Papa, and S. Shenoi, "A Taxonomy of Attacks on the DNP3 Protocol," in *Critical Infrastructure Protection III (IFIP AICT, Vol.\ 311)*. Springer, 2009, pp. 67–81.

[14] Y. Liu, P. Ning, and M. K. Reiter, "False Data Injection Attacks against State Estimation in Electric Power Grids," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2009, pp. 21–32.

[15] T. Kohno, A. Broido, and K. C. Claffy, "Remote Physical Device Fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 2, no. 2, pp. 93–108, 2005.

[16] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A Technique for Physical Device and Device Type Fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 5, pp. 519–532, 2015.

[17] Q. Gu, D. Formby, S. Ji, H. Cam, and R. Beyah, "Fingerprinting for Cyber-Physical System Security: Device Physics Matters Too," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 49–59, Sep. 2018.

[18] S. Jeon, J.-H. Yun, S. Choi, and W.-N. Kim, "Passive Fingerprinting of SCADA in Critical Infrastructure Network without Deep Packet Inspection," arXiv:1608.07679 [cs.CR], Aug. 2016.

[19] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks Against Process Control Systems: Risk Assessment, Detection, and Response," in *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security (ASIACCS)*, 2011, pp. 355–366.

[20] H. Lin, J. Zhuang, Y.-C. Hu, and H. Zhou, "DefRec: Establishing Physical Function Virtualization to Disrupt Reconnaissance of Power Grids' Cyber-Physical Infrastructures," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2020.

[21] H. Lin, Z. T. Kalbarczyk, and R. K. Iyer, "RAINCOAT: Randomization of Network Communication in Power Grid Cyber Infrastructure to Mislead Attackers," *IEEE Transactions on Smart Grid*, vol. 10, no. 5, pp. 4893–4906, 2019.

[22] H. Lin, B. Shrestha, and Y.-C. Hu, "Cyber-Physical Testbed: Case Study to Evaluate Anti-Reconnaissance Approaches on Power Grids' Cyber-Physical Infrastructures," in *Proc. Workshop on Learning from Authoritative Security Experiment Results (LASER)*, 2020.

[23] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an Efficient Website Fingerprinting Defense," in *Computer Security – ESORICS 2016*, 2016.

[24] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, "Website Fingerprinting at Internet Scale," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.

[25] N. Apthorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the Smart Home Private with Smart(er) IoT Traffic Shaping," *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2019.

[26] A. Mehta, M. Alzayat, R. D. Viti, B. B. Brandenburg, P. Druschel, and D. Garg, "Pacer: Comprehensive Network Side-Channel Mitigation in the Cloud," in *Proceedings of the 31st USENIX Security Symposium*, 2022.

[27] A. Sabzi, R. Vora, S. Goswami, M. Seltzer, M. Lécuyer, and A. Mehta, "NetShaper: A Differentially Private Network Side-Channel Mitigation System," in *Proceedings of the 33rd USENIX Security Symposium*, 2024.

[28] A. Sivaraman, S. Subramanian, M. Alizadeh, S. Chole, S.-T. Chuang, Agrawal, Anurag, H. Balakrishnan, T. Edsall, S. Katti, and N. McKeown, "Programmable Packet Scheduling at Line Rate," in *Proc. ACM SIGCOMM Conference*, 2016, pp. 44–57.

[29] A. G. Alcoz, A. Dietmüller, and L. Vanbever, "SP-PIFO: Approximating Push-In First-Out Behaviors using Strict-Priority Queues," in *Proc. 17th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 59–76.

[30] R. Meier, P. Tsankov, V. Lenders, L. Vanbever, and M. Vechev, "NetHide: Secure and Practical Network Topology Obfuscation," in *Proc. 27th USENIX Security Symposium*, 2018, pp. 693–709.

[31] Z. Hu, H. Lin, L. Waind, Y. Qu, G. Chen, and D. Jin, "Industrial Network Protocol Security Enhancement Using Programmable Switches," in *Proceedings of the IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, 2023.

[32] H. Lin, A. Slagell, C. D. Martino, Z. Kalbarczyk, and R. K. Iyer, "Adapting Bro into SCADA: Building a Specification-Based Intrusion Detection System for the DNP3 Protocol," in *Proceedings of the Eighth Annual Cyber Security and Information Intelligence Research Workshop (CSIIRW)*, 2013.

[33] I. N. Fovino, A. Carcano, T. D. L. Murel, A. Trombetta, and M. Masera, "Modbus/DNP3 State-Based Intrusion Detection System," in *2010 24th IEEE International Conference on Advanced Information Networking and Applications (AINA)*, 2010, pp. 729–736.